gpredomics

Interpretable Machine Learning for Omics Data

v0.9.0

Technical Documentation

March 25, 2026

Edi Prifti, Louison Music, Vadim Puller, Jean-Daniel Zucker

IRD — UMMISCO

github.com/predomics/gpredomics

Table of Contents

1 Overview

2 Architecture

3 Data Pipeline

4 Individual & Model Languages

5 Optimization Algorithms

5.1 Genetic Algorithm (GA)

5.2 Beam Search

5.3 MCMC / Bayesian

5.4 Ant Colony Optimization (ACO)

5.5 Simulated Annealing (SA)

5.6 Iterated Local Search (ILS)

5.7 LASSO / Elastic Net

6 Population & FBM Selection

7 Cross-Validation

8 Voting Ensemble

9 Feature Importance

10 GPU Acceleration

11 Metrics & Statistics

12 Configuration Reference

13 Testing & CI

14 Ecosystem

15 Benchmark Results

16 References

1. Overview

gpredomics is a high-performance Rust engine for discovering sparse, interpretable classification models from high-dimensional omics data (metagenomics, transcriptomics, metabolomics). It implements the **BTR (Binary-Ternary-Ratio)** framework where models use discrete coefficients {-1, 0, +1} that clinicians can directly interpret.

The Problem

Standard machine learning on omics data produces black-box models (Random Forest, XGBoost) that achieve high accuracy but cannot be interpreted by domain experts. Feature selection is computationally expensive, models overfit easily on small clinical cohorts, and results vary across random seeds.

The Solution

gpredomics constrains models to use discrete coefficients, producing rules like:

```
Class 1: (Bacteroides + Prevotella) - (Faecalibacterium) ≥ 1
```

This is directly interpretable: if the sum of Bacteroides and Prevotella prevalence minus Faecalibacterium prevalence exceeds 1, predict disease. Seven different optimization algorithms search for these sparse models, and a voting ensemble combines the best ones.

Key Numbers

Metric	Value
Source code	~47,000 lines of Rust
Tests	776 unit + integration tests
Algorithms	7 (GA, Beam, MCMC, ACO, SA, ILS, LASSO)
Languages	5 (Binary, Ternary, Pow2, Ratio, MCMC Generic)
Fitness functions	8 (AUC, sensitivity, specificity, MCC, F1, NPV, PPV, G-mean)
GPU support	Metal (macOS), Vulkan (Linux)

2. Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    predomicsapp (Web UI)                     │
│                   Vue.js + FastAPI + PostgreSQL              │
└──────────────────────────┬───────────────────────────────────┘
                                      │ HTTP/WebSocket
┌──────────────────────────▼───────────────────────────────────┐
│                    gpredomicspy (Python)                     │
│                   PyO3 bindings + clinical                   │
└──────────────────────────┬───────────────────────────────────┘
                                          │ FFI
┌──────────────────────────▼───────────────────────────────────┐
│                                                              │
│                    gpredomics (Rust Core)                    │
│                                                              │
 │  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
 │  │   Data   │  │  Param   │  │Individual│  │Population│  │
 │  │ loading  │  │  config  │  │  (BTR    │  │  (sorted  │  │
 │  │selection │  │  13 sect │  │   model) │  │   models) │  │
 │  └────┬─────┘  └────┬─────┘  └────┬─────┘  └────┬─────┘  │
│       └──────────────┴──────────────┴──────────────┘        │
│                          │                                  │
 │  ┌──────┬──────┬──────┬──────┬──────┬──────┐             │
 │  │ GA   │ Beam │ MCMC │ ACO  │ SA   │ ILS  │LASSO │       │
 │  └──────┴──────┴──────┴──────┴──────┴──────┘             │
│                          │                                  │
 │  ┌──────┐  ┌──────┐  ┌──────┐  ┌──────┐  │
 │  │  CV  │  │Importance│  │Voting│  │ GPU  │  │
 │  │K-fold│  │  MDA     │  │Jury  │  │wgpu  │  │
 │  └──────┘  └──────┘  └──────┘  └──────┘  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Figure 1: gpredomics ecosystem architecture

Source Code Structure

Module	File	Lines	Purpose
Core	`lib.rs`	755	Public API, entry points (`run()` , `run_training()`)
CLI	`main.rs`	232	Binary, signal handling, parameter parsing
Config	`param.rs`	1,568	13 configuration sections with validation
Data	`data.rs`	4,020	Loading, feature selection, preprocessing
Model	`individual.rs`	7,569	BTR model structure, evaluation, metrics
Collection	`population.rs`	4,059	Population management, FBM selection
Algorithms	`ga/beam/mcmc/aco/sa/ils/lasso.rs`	10,369	7 optimization algorithms
Ensemble	`voting.rs`	4,882	Jury selection, voting methods
Validation	`cv.rs`	2,682	Stratified K-fold cross-validation
GPU	`gpu.rs`	2,420	wgpu sparse matrix acceleration
Utilities	`utils.rs`	6,266	Metrics, CI methods, statistics
Export	`csv_report.rs`	735	CSV performance report
Experiment	`experiment.rs`	1,434	Result structure, serialization

3. Data Pipeline

3.1 Input Format

gpredomics expects tab-separated files with features in rows (transposed omics convention):

```
# X.tsv (features × samples)
feature_name    sample1    sample2    sample3    ...
msp_0001        0.023      0.001      0.045      ...
msp_0002        0.0        0.012      0.003      ...

# y.tsv (class labels)
sample_name    class
sample1        0
sample2        1
sample3        0
```

3.2 Feature Selection

Before optimization, features are filtered by statistical significance:

Method	Test	Use Case
`wilcoxon`	Wilcoxon rank-sum	Non-parametric, default for metagenomics
`studentt`	Student's t-test	Parametric, normal distributions
`bayesian_fisher`	Bayesian + Fisher's exact	Bayes factor filtering

Filtering criteria (configurable):

- `feature_minimal_prevalence_pct`: minimum % of samples with non-zero values (default: 10%)
- `feature_maximal_adj_pvalue`: maximum adjusted p-value (default: 0.05)
- `feature_minimal_feature_value`: minimum absolute feature value (default: 0.0)

3.3 Data Types

Type	Code	Transform	Use Case
Raw	`raw`	No transform	Abundance data
Prevalence	`prev`	$x > 0 \rightarrow 1$, else 0	Presence/absence patterns
Log	`log`	$\log_{10}(x + \varepsilon)$	Log-scale abundance

Multiple data types can be combined: `data_type: "raw,prev"` generates separate populations for each.

4. Individual & Model Languages

An **Individual** represents a single sparse classification model. Its key components:

```
pub struct Individual {
    pub features: BTreeMap<usize, i8>,  // feature_idx → coefficient
    pub k: usize,                        // number of active features
    pub fit: f64,                        // penalized fitness (universal)
    pub cls: ClassificationMetrics,      // AUC, sensitivity, specificity, ...
    pub language: u8,                    // model language
    pub data_type: u8,                   // data transformation
    pub hash: u64,                       // deterministic hash for deduplication
}
```

> **BTreeMap for determinism:** Features are stored in a `BTreeMap` (not `HashMap`) to ensure deterministic iteration order. This guarantees identical results across runs with the same seed, regardless of Rust's random HashMap seeding.

4.1 Model Languages

Language	Coefficients	Scoring Formula	Example
Binary	{0, +1}	score = $\Sigma(x_i) \geq t$	A + B + C ≥ 2
Ternary	{-1, 0, +1}	score = $\Sigma(c_i \times x_i) \geq t$	A + B - C ≥ 1
Pow2	{...-4,-2,-1,0,+1,+2,+4...}	score = $\Sigma(2^n \times x_i) \geq t$	4A + 2B - 8C ≥ 3
Ratio	{-1, 0, +1}	score = Σ(pos) / Σ(neg) ≥ t	(A+B) / (C+D) ≥ 1.5
MCMC Generic	continuous β	P(y=1) = σ(a·pos + b·neg + c)	Bayesian posterior

4.2 Classification Metrics

```
pub struct ClassificationMetrics {
    pub auc: f64,                        // Area Under ROC Curve
    pub threshold: f64,                  // Optimal classification threshold
    pub threshold_ci: Option<ThresholdCI>, // Bootstrap confidence interval
    pub sensitivity: f64,                // True Positive Rate
    pub specificity: f64,                // True Negative Rate
    pub accuracy: f64,                   // Overall accuracy
    pub additional: AdditionalMetrics,   // MCC, F1, NPV, PPV, G-mean
}
```

5. Optimization Algorithms

All algorithms share a common interface: they receive preprocessed `Data` and `Param`, and return a `Vec<Population>` of candidate models ranked by fitness.

5.1 Genetic Algorithm (GA)

Population-based **Recombinative**

The GA evolves a population of models through selection, crossover, and mutation over multiple generations. It is the original and most mature algorithm in gpredomics.

Evolutionary Operators

Operator	Description	Parameter
Elite selection	Top models preserved unchanged	`select_elite_pct: 2.0`
Niche selection	Best per ecological niche (language × data_type)	`select_niche_pct: 0.0`
Random selection	Random survival for diversity	`select_random_pct: 2.0`
Crossover	Uniform crossover of feature sets	`mutated_children_pct: 80.0`
Mutation	Random feature add/remove/flip	`mutated_features_pct: 20.0`

Reference: Holland, J.H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

5.2 Beam Search

Systematic **Exhaustive**

Beam Search systematically explores model space level by level, from k=1 features to k=k_stop. At each level, it evaluates all combinations (or the best subset) and selects the Family of Best Models (FBM) to extend.

Methods

- **LimitedExhaustive** (default): Evaluate all C(n,k) combinations, pruning when too many
- **ParallelForward**: Extend existing models by one feature at a time

Reference: Lowerre, B.T. (1976). *The HARPY speech recognition system*. Carnegie Mellon University.

5.3 MCMC / Bayesian Inference

Bayesian | Posterior sampling

Bayesian logistic regression with Metropolis-Hastings sampling. Two modes:

- **Gibbs variable selection** (default, `method: gibbs`): Joint sampling of feature inclusion and coefficients in a single MCMC run, with a sparsity prior P(k) controlling model complexity.
- **Sequential Backward Selection** (`method: sbs`): Nested MCMC at each feature elimination step (original method).

The Gibbs mode includes LASSO pre-screening (top 50 features) and an adaptive BTR model conversion from posterior inclusion probabilities.

References: Metropolis, N. et al. (1953). *Equations of state calculations by fast computing machines*. J. Chem. Phys.; George, E.I. & McCulloch, R.E. (1993). *Variable selection via Gibbs sampling*. JASA.

5.4 Ant Colony Optimization (ACO)

Population-based | Constructive

Ants construct models feature-by-feature using pheromone trails. Higher pheromone on a feature means it was found useful by previous ants. Uses the Max-Min Ant System (MMAS) variant with pheromone bounds to prevent premature convergence.

Key innovation: pheromone is tracked per (feature, coefficient_sign) pair, encoding not just feature importance but also directional association.

Reference: Dorigo, M. & Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.

5.5 Simulated Annealing (SA)

Single-solution | Perturbative

Starts with a random model and iteratively proposes small changes (add/remove/flip one feature). Improvements are always accepted; worse solutions are accepted with probability $e^{-\Delta E/T}$ where T decreases over time (cooling schedule).

Reference: Kirkpatrick, S. et al. (1983). *Optimization by simulated annealing*. Science, 220(4598).

5.6 Iterated Local Search (ILS)

Single-solution | Perturbative

Alternates between local search (greedy hill climbing) and perturbation (random multi-feature changes). The local search finds the nearest local optimum; perturbation escapes it to explore new regions.

Reference: Lourenço, H.R. et al. (2003). *Iterated Local Search*. In Handbook of Metaheuristics, Springer.

5.7 LASSO / Elastic Net

Direct optimization | Convex

L1-regularized regression via coordinate descent along a regularization path ($\alpha_{max} \rightarrow \alpha_{min}$). Produces the mathematically optimal sparse linear model. Continuous LASSO coefficients are converted to discrete BTR coefficients by taking the sign.

Serves as a **mathematical baseline**: how well can an optimal sparse linear model do compared to heuristic BTR models?

References: Tibshirani, R. (1996). *Regression shrinkage and selection via the lasso*. JRSS-B; Friedman, J. et al. (2010). *Regularization paths for generalized linear models via coordinate descent*. JSS.

6. Population & FBM Selection

A `Population` is a sorted collection of `Individual` models. The **Family of Best Models (FBM)** is selected using confidence interval overlap on the fitness metric.

FBM Selection Algorithm

1. Compute Wilson score confidence interval for the best model's AUC
2. Include all models whose AUC overlaps with this interval
3. Remove clones (identical feature sets, using deterministic hashing)
4. Enforce diversity (minimum Jaccard dissimilarity between models)

Diversity Metric

Signed Jaccard dissimilarity between two models A and B:

```
J(A,B) = 1 - |A ∩ B| / |A ∪ B|
```

where intersection considers both feature index AND coefficient sign.

7. Cross-Validation

Stratified K-fold cross-validation with optional double stratification by class and sample annotation (e.g., study site, batch).

Modes

Mode	Parameters	Description
Inner CV	`inner_folds: 5`	Within-fold model selection
Outer CV	`outer_folds: 5`	Performance estimation across folds
Overfit penalty	`overfit_penalty: 0.0`	Penalize train-test performance gap

The FBM is computed across folds: models that appear consistently across multiple folds are ranked higher.

8. Voting Ensemble

The **Jury** selects diverse, high-performing models from the FBM and combines their predictions through voting.

Expert Selection Pipeline

1. **FBM CI filter**: Models within AUC confidence interval of the best
2. **min_perf filter**: Minimum individual AUC (default: 0.5)
3. **min_diversity filter**: Minimum Jaccard distance between experts (default: 5%)

Voting Methods

Method	Rule
Majority	Class = mode of expert predictions
Consensus	Class = unanimous agreement, else reject

Expert Specialization

Experts can be classified based on their per-class performance:

- **PositiveSpecialist**: High sensitivity, moderate specificity → votes only on positive predictions
- **NegativeSpecialist**: High specificity, moderate sensitivity → votes only on negative predictions

- **Balanced**: Both metrics above threshold → votes on all predictions

9. Feature Importance

Five types of feature importance, computed at population or cross-validation level:

Type	Description	Scope
MDA	Mean Decrease in Accuracy (permutation importance)	Population
Prevalence (Pop)	% of FBM models containing the feature	Population
Prevalence (CV)	% of CV folds where feature appears in FBM	Collection
Coefficient	Mean |coefficient| across FBM models	Population
Posterior Prob.	P(feature active) from MCMC posterior	MCMC only

10. GPU Acceleration

Optional GPU acceleration via **wgpu** for fitness evaluation. Supports Metal (macOS), Vulkan (Linux), and DirectX 12 (Windows).

Implementation

The feature matrix X is converted to Compressed Sparse Row (CSR) format. Individual models are encoded as Compressed Sparse Column (CSC) vectors. GPU computes sparse matrix-vector products for all individuals in parallel.

Memory Policies

Policy	Behavior
`Strict`	Respect memory limits strictly, fail if exceeded
`Adaptive`	Balance memory usage and performance (default)
`Performance`	Maximize GPU utilization, may use more memory

11. Metrics & Statistics

11.1 Classification Metrics

Metric	Formula
AUC	Area under ROC curve (trapezoidal integration)
Sensitivity	TP / (TP + FN)
Specificity	TN / (TN + FP)
Accuracy	(TP + TN) / (TP + TN + FP + FN)
MCC	Matthews Correlation Coefficient
F1	2 × PPV × Sensitivity / (PPV + Sensitivity)
PPV	TP / (TP + FP)
NPV	TN / (TN + FN)
G-mean	√(Sensitivity × Specificity)

11.2 Confidence Interval Methods

Method	Use Case
Wilson score (default)	Best for small samples, bounded [0,1]
Wald	Normal approximation, simple but can exceed [0,1]
Agresti-Coull	Adjusted Wald with pseudo-counts
Clopper-Pearson	Exact, conservative (widest intervals)

12. Configuration Reference

Complete parameter file (`param.yaml`) with 13 sections:

```yaml
general:
  seed: 42              # Random seed for reproducibility
  algo: ga              # ga|beam|mcmc|aco|sa|ils|lasso
  language: ter         # bin|ter|pow2|ratio (comma-separated for multi)
  data_type: prev       # raw|prev|log (comma-separated for multi)
  fit: auc              # auc|sensitivity|specificity|mcc|f1|npv|ppv|g_mean
  k_penalty: 0.0001     # Penalty per feature (controls sparsity)
  thread_number: 4      # Parallel threads
  gpu: false            # Enable GPU acceleration
  cv: false             # Enable cross-validation

data:
  X: data/Xtrain.tsv
  y: data/Ytrain.tsv
  Xtest: data/Xtest.tsv
  ytest: data/Ytest.tsv
  features_in_rows: true
  holdout_ratio: 0.2
  feature_selection_method: wilcoxon
  feature_minimal_prevalence_pct: 10
  feature_maximal_adj_pvalue: 0.05

ga:
  population_size: 5000
  max_epochs: 200
  kmin: 1
  kmax: 200

voting:
  vote: true
  method: Majority
  min_perf: 0.5
  min_diversity: 5.0
```

13. Testing & CI

13.1 Test Suite

Category	Count	Coverage
Unit tests (lib)	776	Individual, Population, Data, Utils, CV, Voting, all algorithms
Integration (GA e2e)	24	Full pipeline on Qin2014 dataset
Integration (Beam e2e)	28	Beam search on Qin2014
Integration (MCMC e2e)	15	MCMC posterior + SBS on Qin2014
Consistency	9	Serialization, cross-version compatibility
CSV Report	5	Export format validation

13.2 CI Pipeline

GitHub Actions (`.github/workflows/rust.yml`):

1. **Lint**: `cargo fmt --all --check` + `cargo clippy --all-targets -D warnings`
2. **Build & Test**: `cargo build` + `cargo test` (30-minute timeout)

14. Ecosystem

Component	Language	Repository	Purpose
gpredomics	Rust	predomics/gpredomics	Core engine (this document)
gpredomicspy	Rust + Python	predomics/gpredomicspy	Python bindings via PyO3
predomicsapp	Vue.js + FastAPI	predomics/predomicsapp	Web application
GpredomicsR	R	(coming soon)	R package wrapper

15. Benchmark Results

Dataset: **Qin et al. (2014)** — Liver cirrhosis metagenomics. 1,980 features (MSPs), 180 training samples, 30 test samples.

	Algorithm	Test AUC	Model Size (k)	Time	Type
🥇	**Beam Search**	**0.947**	8	0.5s	Systematic
🥈	**LASSO / Elastic Net**	0.882	70	0.1s	Direct optimization
🥉	**Simulated Annealing**	0.856	40	1s	Single-solution
	Genetic Algorithm	0.855	28	20s	Population, multi-language
	Ant Colony Optimization	0.853	55	15s	Population, constructive
	Iterated Local Search	0.813	**22**	**0.1s**	Single-solution
	MCMC / Bayesian	0.738	35	5s	Bayesian (posterior)

> **Interpretation:** Beam Search achieves the best test AUC (0.947) with the smallest models (k=8) because it systematically explores all small model combinations. Population-based methods (GA, ACO) find diverse model sets. Single-solution methods (SA, ILS) are the fastest. LASSO provides the mathematical baseline.

16. References

Methodology

1. Prifti, E. et al. (2020). Interpretable and accurate prediction models for metagenomics data. *GigaScience*, 9(3). doi:10.1093/gigascience/giaa010
2. Qin, N. et al. (2014). Alterations of the human gut microbiome in liver cirrhosis. *Nature*, 513(7516). doi:10.1038/nature13568

Algorithms

1. Holland, J.H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
2. Lowerre, B.T. (1976). *The HARPY speech recognition system*. CMU.
3. Metropolis, N. et al. (1953). Equations of state calculations by fast computing machines. *J. Chem. Phys.*, 21(6).
4. George, E.I. & McCulloch, R.E. (1993). Variable selection via Gibbs sampling. *JASA*, 88(423).
5. Dorigo, M. & Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.
6. Kirkpatrick, S. et al. (1983). Optimization by simulated annealing. *Science*, 220(4598).

7. Lourenço, H.R. et al. (2003). Iterated Local Search. In *Handbook of Metaheuristics*, Springer.
8. Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *JRSS-B*, 58(1).
9. Friedman, J. et al. (2010). Regularization paths for generalized linear models via coordinate descent. *JSS*, 33(1).

Statistics

1. Wilson, E.B. (1927). Probable inference, the law of succession, and statistical inference. *JASA*, 22(158).
2. Agresti, A. & Coull, B.A. (1998). Approximate is better than "exact" for interval estimation of binomial proportions. *The American Statistician*, 52(2).

---

gpredomics v0.9.0 — Generated March 25, 2026 — github.com/predomics/gpredomics